

Student Record Management System

Evaluated Course Project Report

Course: Python Essentials

Branch: CSE (AIML)

Submitted by: Vrushabha BN

Platform: VITyarthi

GitHub Repository:

<https://github.com/Vrushabha056/student-record-system>

1. Introduction

This report documents the design, implementation, and evaluation of a Student Record Management System, developed as the evaluated course project for the Python Essentials course on the VITyarthi platform. The project applies core Python concepts — functions, data structures, file handling, exception handling, and modular program design — to build a working, testable command-line application.

2. Problem Statement

Educational institutions and individual students often need a simple way to keep track of student records — names, ages, courses, and marks — without relying on spreadsheets that are easy to accidentally overwrite, or full database systems that are overkill for small-scale use. This project addresses that gap with a lightweight, dependency-free command-line tool that allows adding, viewing, updating, deleting, and searching student records, with data persisted locally between sessions.

Target Users

- Individual students or small course instructors who need to track a small set of student records without setting up a database.
- Learners studying Python who want a reference example of a modular, testable CLI application.

3. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output structure:

Module	Responsibility	Input	Output
Data Persistence (storage.py)	Load/save records to disk	In-memory list / JSON file	Updated JSON file / loaded list
Record Management (operations.py)	Add, update, delete, search records	Student fields, search term	Updated list, success/error message
CLI (cli.py, main.py)	Menu-driven user interaction	Menu choice, field values	Formatted console output

Core Features

- Add a new student record (ID, Name, Age, Course, Marks) with validation.
- View all student records in a formatted table.
- Update an existing record, changing only the fields provided.
- Delete a record, with confirmation before removal.
- Search for a record by ID or partial name match.

- Automatic, persistent storage to a local JSON file.

Logical Workflow

On startup, the program loads existing records from students.json. The user is shown a menu and selects an operation. Add/Update/Delete operations modify the in-memory list and then immediately persist the change to disk. The loop continues until the user selects Exit.

4. Non-Functional Requirements

Requirement	How it is addressed
Reliability	Try/except blocks around file reads and numeric input prevent crashes on bad input or a corrupted data file.
Usability	Simple numbered menu, clear prompts, and confirmation before destructive actions (delete).
Maintainability	Code is split into single-responsibility modules (model, storage, operations, CLI), making changes localized and safe.
Error Handling Strategy	All user input is validated (ID uniqueness, age range, marks range) before being accepted, with descriptive error messages.
Resource Efficiency	Uses only Python's standard library; the JSON file is small and read/written only when needed, not on every keystroke.

5. System Architecture

The system follows a layered architecture with a clear separation of concerns. The CLI layer handles all user interaction and delegates work to the Record Management layer, which contains the core business logic. The Record Management layer uses the Data Model layer to build and validate records, and the Data Persistence layer to read from and write to the JSON data file. No layer directly accesses the file system except the Data Persistence module, keeping storage concerns isolated.

Module Overview

- **student.py** — Data model: defines the structure of a student record and provides validation and formatting helpers.
- **storage.py** — Data Persistence module: loads records from and saves records to `students.json`.
- **operations.py** — Record Management module: implements add, update, delete, search, and lookup logic.
- **cli.py** — CLI module: displays the menu, collects input, and calls into the other modules.
- **main.py** — Entry point that starts the CLI loop.
- **tests/test_operations.py** — Unit tests covering the Record Management module.

The full system architecture diagram, process flow diagram, use case diagram, class/component diagram, and sequence diagram are provided in **diagrams.md** in the project's GitHub repository (rendered natively by GitHub via Mermaid). They are summarized below.

6. Design Diagrams

6.1 System Architecture Diagram

User → CLI module → Record Management module → Data Model module. The Record Management and CLI modules both interact with the Data Persistence module, which reads/writes `students.json`.

6.2 Process Flow / Workflow Diagram

Start → Load records from file → Display menu → Perform selected operation (Add/View/Update/Delete/Search) → Save changes if applicable → Return to menu → Exit when chosen.

6.3 Use Case Diagram

A single actor (User) interacts with five use cases: Add Student, View Students, Update Student, Delete Student, Search Student.

6.4 Class / Component Diagram

Since the project uses functions and dictionaries rather than classes, the component diagram represents each module's public functions and their dependencies: `cli.py` depends on `operations.py`, `storage.py`, and `student.py`; `operations.py` depends on `student.py`.

6.5 Sequence Diagram — Add Student

User selects 'Add Student' → CLI prompts for fields → CLI calls `operations.add_student()` → `operations` validates via `student.py` and builds the record → CLI calls `storage.save_students()` → result is written to `students.json` → CLI displays the outcome to the user.

(Full diagrams with visual boxes and arrows are available in `diagrams.md` in the GitHub repository.)

7. Design Decisions & Rationale

- **Dictionaries over classes:** chosen to keep the implementation approachable at a Python Essentials level while still demonstrating clean data modeling.
- **JSON over a database:** the project's scope (single user, small dataset) does not need a database engine; JSON is human-readable and needs no setup.
- **Separation into modules:** isolating persistence, business logic, and the CLI means each part can be tested and modified independently — e.g., the storage mechanism could be swapped for a database later without touching `operations.py`.
- **Validation inside `operations.py` rather than `cli.py`:** keeps business rules (valid age/marks ranges, unique IDs) independent of how the user interface is built, making the logic reusable.

8. Implementation Details

Key implementation points:

- Records are represented as dictionaries with keys: `id`, `name`, `age`, `course`, `marks`.
- `storage.py` uses Python's built-in `json` module to serialize/deserialize the record list, with `try/except` handling for a missing or corrupted data file.
- `operations.py` exposes pure functions that take the current list of students and return a (success, message) tuple, making them easy to unit test without any input/output side effects.
- `cli.py` is the only module that calls `input()` and `print()`, keeping all user-facing text in one place.
- Input validation (ValueError handling) guards against non-numeric entries for ID, Age, and Marks.

Example — core validation logic (`operations.py`):

```
def add_student(students, student_id, name, age, course, marks):
    if find_student_by_id(students, student_id):
        return False, "A student with this ID already exists."
    if not validate_age(age):
```

```
    return False, "Age must be between 5 and 100."
    ...
```

9. Screenshots / Results

[Insert screenshots here before final submission: the main menu, adding a student, viewing the list, updating a record, deleting a record, and search results. Screenshots can be taken directly from your terminal after running `python main.py`.]

10. Testing Approach

The Record Management module is covered by 7 automated unit tests using Python's built-in unittest framework, located in tests/test_operations.py. Tests run against an in-memory list (no file I/O), making them fast and independent of any existing data file.

- test_add_student_success — verifies a valid student is added.
- test_add_student_duplicate_id — verifies duplicate IDs are rejected.
- test_add_student_invalid_marks — verifies out-of-range marks are rejected.
- test_update_student — verifies a field can be updated correctly.
- test_delete_student — verifies a record can be removed.
- test_search_by_name — verifies case-insensitive partial name search.
- test_find_student_by_id_not_found — verifies a lookup miss returns None.

All 7 tests pass. Tests can be run with: `python -m unittest discover -s tests`

Manual validation testing was also performed by running the CLI end-to-end and confirming correct behavior for invalid input (non-numeric ID/age/marks, out-of-range values, non-existent IDs).

11. Challenges Faced

- Deciding how to split a single-file program into modules without introducing circular imports — resolved by keeping operations.py independent of cli.py, with dependencies flowing in one direction only.
- Handling partial updates (where a user leaves a field blank) required distinguishing between 'no change requested' and 'invalid value provided', particularly for numeric fields.
- Setting up Git and GitHub for the first time, including resolving a missing Git installation and configuring author identity before the first commit.

12. Learnings & Key Takeaways

- Practical experience structuring a Python program into reusable, testable modules rather than one large script.
- Hands-on use of file handling (JSON) for data persistence.
- Writing and running unit tests with unittest, and understanding why testing pure logic functions (separate from input/output) is easier.
- End-to-end use of Git and GitHub for version control, including committing, pushing, and maintaining a clean repository with .gitignore.

13. Future Enhancements

- Replace JSON storage with a SQLite database for larger datasets and safer concurrent access.
- Add a graphical user interface (e.g., with Tkinter) as an alternative to the CLI.
- Add sorting and reporting features, such as class average marks or top-performing students.
- Add multi-user support with basic authentication.
- Export records to CSV or PDF for sharing outside the application.

14. References

- Python Software Foundation. Python 3 Documentation — json, os, unittest modules. <https://docs.python.org/3/>
- Git documentation. <https://git-scm.com/doc>
- GitHub Docs — Mermaid diagram support. <https://docs.github.com/>
- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (course-provided document).